

Grundlagen der Programmierung: 12. Klausurvorbereitung II

Tutoren:

Daniel Grillmeyer, Ludwig Friedl, Jan Kerber, Jennifer Muck

1. Beispielaufgaben aus vergangenen Klausuren (Fortsetzung)
 1. Java Stack
2. Anspruchsvolle Aufgaben zur Vertiefung
 1. Java Codeverständnis
 2. Java Iterable Vector4D
3. Lösung Programmieraufgabe 4
4. Fragestunde

Klausurbeispielaufgaben

JAVA STACK

Java Stack – 18 Punkte

In dieser Aufgabe implementieren Sie einen Stack (Stapel) zu Ende. Ein Stack ist eine Datenstruktur, bei der Elemente immer nur oben auf den Stapel gelegt oder von oben abgenommen werden können.

Hinweis: Gehen Sie in den Teilaufgaben davon aus, dass die vorherigen Teilaufgaben korrekt gelöst wurden.

a) Implementieren Sie den Konstruktor der Klasse Stack. Der Parameter `size` gibt die Größe des internen Arrays an. Prüfen Sie vor Erstellung und Zuweisung des neuen Arrays an die Instanzvariable `values`, ob die angegebene Größe des Stacks größer 0 ist. Sollte der übergebene Wert größer sein, wird `size` verwendet, ansonsten wird eine Standardgröße von 10 angenommen.

2 Punkte

Java Stack – 18 Punkte

```
1 public class Stack {  
2     protected int[] values;  
3     protected int currentIndex = 0;  
4  
5     public Stack(int size) {  
6  
7  
8  
9  
10  
11  
12  
13  
14  
15 }  
16 ...
```

Java Stack – 18 Punkte

```
1 public class Stack {  
2     protected int[] values;  
3     protected int currentIndex = 0;  
4  
5     public Stack(int size) {  
6         if (size <= 0) {  
7             values = new int[10];  
8         } else {  
9             values = new int[size];  
10        }  
11    }  
12    ...  
13  
14  
15  
16
```

Java Stack – 18 Punkte

b) Implementieren Sie die drei Methoden push, pop und peek. Diese sind Teil der Klasse Stack und arbeiten immer auf dem obersten Integer des Stacks, der durch die Instanzvariable currentIndex repräsentiert ist.

- push: Fügt den Integer oben auf dem Stack ein und erhöht currentIndex um eins.
- pop: Nimmt den obersten Integer vom Stack, verringert currentIndex um eins und gibt den entnommenen Integer zurück.
- peek: Gibt den obersten Integer auf dem Stack zurück, ohne den Stack zu verändern.

3 Punkte

Java Stack – 18 Punkte

```
1 public class Stack {  
2     ...  
3     public void push (int value) {  
4  
5  
6     }  
7     public int pop() {  
8  
9  
10    }  
11    public int peek() {  
12  
13  
14  
15    }  
16    ...
```

Java Stack – 18 Punkte

```
1 public class Stack {  
2     ...  
3     public void push (int value) {  
4         values[currentIndex++] = value;  
5     }  
6  
7     public int pop() {  
8         return values[--currentIndex];  
9     }  
10  
11    public int peek() {  
12        return values[currentIndex-1];  
13    }  
14  
15  
16    ...
```

Java Stack – 18 Punkte

c) Implementieren Sie ein Enum Operator, um Rechenoperationen einer Stack Maschine zu repräsentieren. Das Enum soll die zwei Werte ADD für Addition und MUL für Multiplikation enthalten.

2 Punkte

1
2
3
4
5

c) Implementieren Sie ein Enum Operator, um Rechenoperationen einer Stack Maschine zu repräsentieren. Das Enum soll die zwei Werte ADD für Addition und MUL für Multiplikation enthalten.

2 Punkte

```
1 public enum Operator {  
2     ADD, MUL;  
3 }  
4  
5
```

d) Im Folgenden ist die Klasse `StackMachine` gegeben. Implementieren Sie die Methode `calculate` fertig. Diese bekommt einen Operator (`ADD` oder `MUL`) übergeben und führt die entsprechende Operation aus. Die obersten beiden Werte werden vom Stack entnommen und danach die Operation auf den beiden Werten ausgeführt. Das Ergebnis wird zurück auf den Stack geschrieben. Für den Fall, dass die Operation unbekannt ist, darf sich der Stack nicht verändern und eventuell entnommene Werte müssen in der richtigen Reihenfolge zurück geschrieben werden.

5 Punkte

Java Stack – 18 Punkte

```
1 public class StackMachine {  
2     private Stack stack;  
3  
4     public StackMachine(Stack stack) {  
5         this.stack = stack;  
6     }  
7  
8     public void calculate(Operator operation) {  
9  
10  
11  
12  
13  
14  
15     }  
16 ...
```

Java Stack – 18 Punkte

```
1 public class StackMachine {
2     private Stack stack;
3
4     public StackMachine(Stack stack) {
5         this.stack = stack;
6     }
7
8     public void calculate(Operator operation) {
9         int left = stack.pop();
10        int right = stack.pop();
11        switch (operation) {
12            case ADD: stack.push(left + right); break;
13            case MUL: stack.push(left * right); break;
14            default: stack.push(right); stack.push(left); break;
15        }
16    ...
```

Java Stack – 18 Punkte

e) Um mehrere Operationen hintereinander auszuführen, implementieren Sie die Methode `run`. Diese bekommt ein Array aus Operationen übergeben, die nacheinander ausgeführt werden sollen. Das endgültige Ergebnis auf dem Stack wird zurückgegeben, ohne es vom Stack zu löschen.

2 Punkte

```
1  ...
2      public int run (Operator[] program) {
3
4
5
6
7
8
9
10 }
```

Java Stack – 18 Punkte

e) Um mehrere Operationen hintereinander auszuführen, implementieren Sie die Methode `run`. Diese bekommt ein Array aus Operationen übergeben, die nacheinander ausgeführt werden sollen. Das endgültige Ergebnis auf dem Stack wird zurückgegeben, ohne es vom Stack zu löschen.

2 Punkte

```
1  ...
2      public int run (Operator[] program) {
3          for (Operator o: program) {
4              calculate(o);
5          }
6          return stack.peek();
7      }
8
9
10
```

Java Stack – 18 Punkte

- f) Implementieren Sie die Klasse `Fibonacci`, die von der Klasse `Stack` erbt, fertig. In der Methode `calculate` sollen die Fibonacci-Zahlen *rekursiv* berechnet werden, bis der Stack keinen Platz mehr bietet. Die Zahlen berechnen sich aus der Summe der beiden obersten Werte auf dem Stack. Die Summe wird auf den Stack geschrieben. Beachten Sie, dass alle Werte auf dem Stack erhalten bleiben müssen. Sie können den Stack über die drei Methoden (`push`, `pop` und `peek`) oder direkt verändern. Zu Beginn liegt immer zweimal der Wert eins auf dem Stack. Die Methode soll die letzte Fibonacci-Zahl zurückgeben.

Hinweis: Arrays besitzen das Attribut `length`, das die maximale Kapazität enthält.

4 Punkte

Java Stack – 18 Punkte

```
1 public class Fibonacci {
2     public Fibonacci(int size) {
3         super(size);
4         push(1);
5         push(1);
6     }
7
8     public int calculate() {
9
10
11
12
13
14
15     }
16 }
```

Java Stack – 18 Punkte

```
1 public class Fibonacci extends Stack {
2     public Fibonacci(int size) {
3         super(size);
4         push(1);
5         push(1);
6     }
7
8     public int calculate() {
9         if (currentIndex >= values.length) {
10            return peek();
11        } else {
12            push(values[currentIndex-1] + values[currentIndex-2]);
13            return calculate();
14        }
15    }
16 }
```

Anspruchsvolle Aufgaben zur Vertiefung

JAVA CODEVERSTÄNDNIS

Java Codeverständnis – 12 Punkte

Gegeben ist eine Klasse namens `ArgExecutor`. Sie nimmt einen `String`, der mehrere Argumente mit dazugehörigen Parametern spezifiziert, zerlegt diesen `String` und führt dann, je nach Eingabe, unterschiedliche Berechnungen aus. Die Klasse ist eine sogenannte *Utility*-Klasse, d.h. alle Methoden der Klasse sind statisch, daher bedarf sie keiner Instanziierung.

Haupteinstiegspunkt von `ArgExecutor` ist die Methode:

`public static double` `parse(String arg);`

Der Parameter `arg` ist der Argument-`String`, der in der jeweiligen Aufgabenstellung spezifiziert ist. Ihre Aufgabe ist die Berechnung des Rückgabewertes der Methode in Abhängigkeit von `arg`. Die vollständige Klassendefinition von `ArgExecutor` finden Sie am Ende der Aufgabe auf einer separaten Seite.

Java Codeverständnis – 12 Punkte

Im Code werden einige Funktionen aus der Java-API aufgerufen. Sie finden folgend Auszüge aus der Java-API-Dokumentation zur Erklärung einiger dieser Funktionen:

<i>String</i>	<code>String[] split(String regex)</code>	Splits this string around matches of the given regular expression.
<i>Arrays</i>	<code>static <T> T[] copyOfRange(T[] original, int from, int to)</code>	Copies the specified range of the specified array into a new array.
<i>HashMap</i> <K,V>	<code>void put(K key, V value)</code>	Associates the specified value with the specified key in this map.
<i>HashMap</i> <K,V>	<code>V get(Object key)</code>	Returns the value to which the specified key is mapped, or null if this map contains no mapping for the key.
<i>HashMap</i> <K,V>	<code>boolean containsKey(Object key)</code>	Returns true if this map contains a mapping for the specified key.

Hinweis: Sie können optional für die einzelnen Parameter Zwischenergebnisse aufschreiben, um Teilpunkte bei falschen Lösungen zu erhalten

Hinweis 2: Wenn Sie in dieser Aufgabe stecken bleiben, bearbeiten Sie am besten erst die anderen Aufgaben dieser Klausur und kehren dann zu dieser Aufgabe zurück.

Java Codeverständnis – 12 Punkte

```
1 public final class ArgExecuter {
2     static HashMap<String, List<String>> map =
3         new HashMap<String, List<String>>();
4
5     public static double parse(String arg) {
6         map.clear();
7         // Gibt die durch Leerzeichen getrennten Teilstrings zurueck.
8         // Die Leerzeichen werden entfernt.
9         String[] args = arg.split(" ");
10        int ki = 0;
11        for (int i = 1; i < args.length; i++)
12            if (args[i].startsWith("-"))
13                ki = parseParams(args, i, ki);
14        parseParams(args, args.length, ki);
15        return res();
16    }
```

Java Codeverständnis – 12 Punkte

```
17 private static int parseParams(String[] args, int i, int ki) {
18     String p = args[ki];
19     putOrAppend(p, Arrays.copyOfRange(args,
20         ki + 1 /*copy start, inclusive*/, i /*copy end, exclusive*/));
21     return i;
22 }
23
24 private static void putOrAppend(String k, String[] args) {
25     List<String> argL;
26     if (map.containsKey(k)) { argL = map.get(k); }
27     else { argL = new ArrayList<String>(); }
28     argL.addAll(Arrays.asList(args));
29     map.put(k, argL);
30 }
```

Java Codeverständnis – 12 Punkte

```
31 private static double res() {  
32     double a = 0;  
33     if (map.containsKey("-a"))  
34         for (String arg : map.get("-a"))  
35             a += Double.parseDouble(arg);  
36     if (map.containsKey("-m"))  
37         for (String arg : map.get("-m"))  
38             a *= Double.parseDouble(arg);  
39     double e = 1;  
40     if (map.containsKey("-e")) {  
41         for (String arg : map.get("-e"))  
42             e *= Double.parseDouble(arg);  
43         a *= Math.pow(10, e);  
44     }  
45     return a;  
46 }  
47 }
```

Java Codeverständnis – 12 Punkte

```
1 public final class ArgExecuter {
2     static HashMap<String, List<String>> map =
3         new HashMap<String, List<String>>();
4
5     public static double parse(String arg) {
6         map.clear();
7         // Gibt die durch Leerzeichen getrennten
8         // Teilstrings zurueck.
9         // Die Leerzeichen werden entfernt.
10        String[] args = arg.split(" ");
11        int ki = 0;
12        for (int i = 1; i < args.length; i++)
13            if (args[i].startsWith("-"))
14                ki = parseParams(args, i, ki);
15        parseParams(args, args.length, ki);
16        return res();
17    }
18
19    private static int parseParams(String[] args,
20                                  int i, int ki) {
21        String p = args[ki];
22        putOrAppend(p, Arrays.copyOfRange(args,
23            ki + 1 /*copy start, inclusive*/,
24            i /*copy end, exclusive*/));
25        return i;
26    }
```

```
27 private static void putOrAppend(String k, String[] args) {
28     List<String> argL;
29     if (map.containsKey(k)) { argL = map.get(k); }
30     else { argL = new ArrayList<String>(); }
31     argL.addAll(Arrays.asList(args));
32     map.put(k, argL);
33 }
34
35 private static double res() {
36     double a = 0;
37     if (map.containsKey("-a"))
38         for (String arg : map.get("-a"))
39             a += Double.parseDouble(arg);
40     if (map.containsKey("-m"))
41         for (String arg : map.get("-m"))
42             a *= Double.parseDouble(arg);
43     double e = 1;
44     if (map.containsKey("-e")) {
45         for (String arg : map.get("-e"))
46             e *= Double.parseDouble(arg);
47         a *= Math.pow(10, e);
48     }
49     return a;
50 }
51 }
52 }
```

Java Codeverständnis – 12 Punkte

```
1 public final class ArgExecuter {
2     static HashMap<String, List<String>> map =
3         new HashMap<String, List<String>>();
4
5     public static double parse(String arg) {
6         map.clear();
7         // Gibt die durch Leerzeichen getrennten
8         // Teilstrings zurueck.
9         // Die Leerzeichen werden entfernt.
10        String[] args = arg.split(" ");
11        int ki = 0;
12        for (int i = 1; i < args.length; i++)
13            if (args[i].startsWith("-"))
14                ki = parseParams(args, i, ki);
15        parseParams(args, args.length, ki);
16        return res();
17    }
18
19    private static int parseParams(String[] args,
20                                  int i, int ki) {
21        String p = args[ki];
22        putOrAppend(p, Arrays.copyOfRange(args,
23            ki + 1 /*copy start, inclusive*/,
24            i /*copy end, exclusive*/));
25        return i;
26    }
```

```
27 private static void putOrAppend(String k, String[] args) {
28     List<String> argL;
29     if (map.containsKey(k)) { argL = map.get(k); }
30     else { argL = new ArrayList<String>(); }
31     argL.addAll(Arrays.asList(args));
32     map.put(k, argL);
33 }
34
35 private static double res() {
36     double a = 0;
37     if (map.containsKey("-a"))
38         for (String arg : map.get("-a"))
39             a += Double.parseDouble(arg);
40     if (map.containsKey("-m"))
41         for (String arg : map.get("-m"))
42             a *= Double.parseDouble(arg);
43     double e = 1;
44     if (map.containsKey("-e")) {
45         for (String arg : map.get("-e"))
46             e *= Double.parseDouble(arg);
47         a *= Math.pow(10, e);
48     }
49     return a;
50 }
51 }
```

a) Eingabe: `parse("-a 2 4 3 -m 1");`

Java Codeverständnis – 12 Punkte

a) Eingabe:

`parse("-a 2 4 3 -m 1");`

4 Punkte

Zwischenergebnisse:

Rückgabewert:

Java Codeverständnis – 12 Punkte

a) Eingabe:

`parse("-a 2 4 3 -m 1");`

4 Punkte

Zwischenergebnisse:

```
parseParams(args, 4, 0)
putOrAppend("-a", {"2", "4", "3"})
parseParams(args, 6, 4)
putOrAppend("-m", {"1"})
```

Rückgabewert:

Java Codeverständnis – 12 Punkte

a) Eingabe:

`parse("-a 2 4 3 -m 1");`

4 Punkte

Zwischenergebnisse:

```
parseParams(args, 4, 0)
putOrAppend("-a", {"2", "4", "3"})
parseParams(args, 6, 4)
putOrAppend("-m", {"1"})
```

Rückgabewert:

9.0

Java Codeverständnis – 12 Punkte

```
1 public final class ArgExecuter {
2     static HashMap<String, List<String>> map =
3         new HashMap<String, List<String>>();
4
5     public static double parse(String arg) {
6         map.clear();
7         // Gibt die durch Leerzeichen getrennten
8         // Teilstrings zurueck.
9         // Die Leerzeichen werden entfernt.
10        String[] args = arg.split(" ");
11        int ki = 0;
12        for (int i = 1; i < args.length; i++)
13            if (args[i].startsWith("-"))
14                ki = parseParams(args, i, ki);
15        parseParams(args, args.length, ki);
16        return res();
17    }
18
19    private static int parseParams(String[] args,
20                                   int i, int ki) {
21        String p = args[ki];
22        putOrAppend(p, Arrays.copyOfRange(args,
23            ki + 1 /*copy start, inclusive*/,
24            i /*copy end, exclusive*/));
25        return i;
26    }
```

```
27 private static void putOrAppend(String k, String[] args) {
28     List<String> argL;
29     if (map.containsKey(k)) { argL = map.get(k); }
30     else { argL = new ArrayList<String>(); }
31     argL.addAll(Arrays.asList(args));
32     map.put(k, argL);
33 }
34
35 private static double res() {
36     double a = 0;
37     if (map.containsKey("-a"))
38         for (String arg : map.get("-a"))
39             a += Double.parseDouble(arg);
40     if (map.containsKey("-m"))
41         for (String arg : map.get("-m"))
42             a *= Double.parseDouble(arg);
43     double e = 1;
44     if (map.containsKey("-e")) {
45         for (String arg : map.get("-e"))
46             e *= Double.parseDouble(arg);
47     }
48     a *= Math.pow(10, e);
49     return a;
50 }
51 }
```

b) Eingabe:
`parse("-a 1 -m 2 2 -a 1.5");`

Java Codeverständnis – 12 Punkte

b) Eingabe:

`parse("-a 1 -m 2 2 -a 1.5");`

4 Punkte

Zwischenergebnisse:

Rückgabewert:

Java Codeverständnis – 12 Punkte

b) Eingabe:

```
parse("-a 1 -m 2 2 -a 1.5");
```

4 Punkte

Zwischenergebnisse:

```
parseParams(args, 2, 0)
putOrAppend("-a", {"1"})
parseParams(args, 5, 2)
putOrAppend("-m", {"2", "2"})
parseParams(args, 7, 5)
putOrAppend("-a", {"1.5"})
```

Rückgabewert:

Java Codeverständnis – 12 Punkte

b) Eingabe:

```
parse("-a 1 -m 2 2 -a 1.5");
```

4 Punkte

Zwischenergebnisse:

```
parseParams(args, 2, 0)
putOrAppend("-a", {"1"})
parseParams(args, 5, 2)
putOrAppend("-m", {"2", "2"})
parseParams(args, 7, 5)
putOrAppend("-a", {"1.5"})
```

Rückgabewert:

10.0

Java Codeverständnis – 12 Punkte

```
1 public final class ArgExecuter {
2     static HashMap<String, List<String>> map =
3         new HashMap<String, List<String>>();
4
5     public static double parse(String arg) {
6         map.clear();
7         // Gibt die durch Leerzeichen getrennten
8         // Teilstrings zurueck.
9         // Die Leerzeichen werden entfernt.
10        String[] args = arg.split(" ");
11        int ki = 0;
12        for (int i = 1; i < args.length; i++)
13            if (args[i].startsWith("-"))
14                ki = parseParams(args, i, ki);
15        parseParams(args, args.length, ki);
16        return res();
17    }
18
19    private static int parseParams(String[] args,
20                                  int i, int ki) {
21        String p = args[ki];
22        putOrAppend(p, Arrays.copyOfRange(args,
23            ki + 1 /*copy start, inclusive*/,
24            i /*copy end, exclusive*/));
25        return i;
26    }
```

```
27 private static void putOrAppend(String k, String[] args) {
28     List<String> argL;
29     if (map.containsKey(k)) { argL = map.get(k); }
30     else { argL = new ArrayList<String>(); }
31     argL.addAll(Arrays.asList(args));
32     map.put(k, argL);
33 }
34
35 private static double res() {
36     double a = 0;
37     if (map.containsKey("-a"))
38         for (String arg : map.get("-a"))
39             a += Double.parseDouble(arg);
40     if (map.containsKey("-m"))
41         for (String arg : map.get("-m"))
42             a *= Double.parseDouble(arg);
43     double e = 1;
44     if (map.containsKey("-e")) {
45         for (String arg : map.get("-e"))
46             e *= Double.parseDouble(arg);
47         a *= Math.pow(10, e);
48     }
49     return a;
50 }
51 }
```

c) Eingabe: `parse("-e 2 -a 1 -m 1.5 -a 1 -m 2 -e 2");`

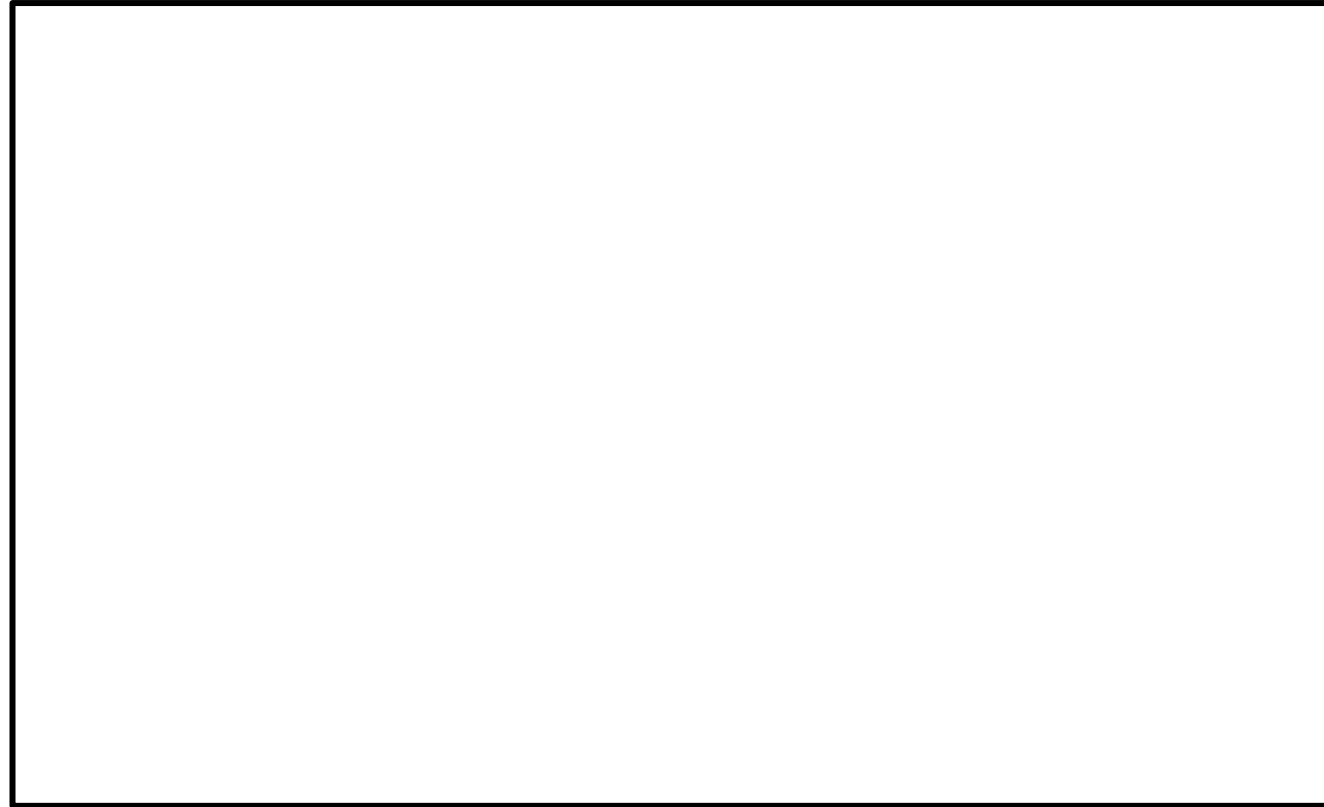
Java Codeverständnis – 12 Punkte

c) Eingabe:

`parse("-e 2 -a 1 -m 1.5 -a 1 -m 2 -e 2");`

4 Punkte

Zwischenergebnisse:



Rückgabewert:

Java Codeverständnis – 12 Punkte

c) Eingabe:

```
parse("-e 2 -a 1 -m 1.5 -a 1 -m 2 -e 2");
```

4 Punkte

Zwischenergebnisse:

```
parseParams(args, 2, 0)
putOrAppend("-e", {"2"})
parseParams(args, 4, 2)
putOrAppend("-a", {"1"})
parseParams(args, 6, 4)
putOrAppend("-m", {"1.5"})
parseParams(args, 8, 6)
putOrAppend("-a", {"1"})
parseParams(args, 10, 8)
putOrAppend("-m", {"2"})
parseParams(args, 12, 10)
putOrAppend("-e", {"2"})
```

Rückgabewert:

Java Codeverständnis – 12 Punkte

c) Eingabe:

```
parse("-e 2 -a 1 -m 1.5 -a 1 -m 2 -e 2");
```

4 Punkte

Zwischenergebnisse:

```
parseParams(args, 2, 0)
putOrAppend("-e", {"2"})
parseParams(args, 4, 2)
putOrAppend("-a", {"1"})
parseParams(args, 6, 4)
putOrAppend("-m", {"1.5"})
parseParams(args, 8, 6)
putOrAppend("-a", {"1"})
parseParams(args, 10, 8)
putOrAppend("-m", {"2"})
parseParams(args, 12, 10)
putOrAppend("-e", {"2"})
```

Rückgabewert:

```
60000.0
```

Anspruchsvolle Aufgaben zur Vertiefung

JAVA ITERABLE VECTOR4D

Java Iterable Vector4D – 10 Punkte

Es ist eine Klasse namens Vector4D teilweise vorimplementiert. Die Klasse hat vier Gleitkommazahl-Attribute (in diesem Fall vom Typ Double), welche die Vektorkomponenten x , y , z und w repräsentieren. Auch schon implementiert sind ein Konstruktor und Getter- und Setter-Methoden.

Diese Klasse soll erweitert werden. Die Vektorkomponenten sollen in einer **for**-Schleife durchlaufbar sein. Ein Beispiel liefert folgender Code:

```
1 Vector4D v = new Vector4D(1.0, 2.0, 3.0, 4.0);
2 for (Double d : v) {
3     // d hat im ersten Durchlauf den Wert 1.0 (x), dann 2.0 (y),
4     // dann 3.0 (z), dann 4.0 (w). Dann bricht die Schleife ab.
5 }
```

Damit dies möglich ist, muss Vector4D das generische Interface Iterable<T> und den dazugehörigen Iterator vom Typ Iterator<T> implementieren.

Java Iterable Vector4D – 10 Punkte

Das Interface `Iterable<T>` erfordert von der implementierenden Klasse die Implementierung der Methode `Iterator<T> iterator()`.

Die Java-API Spezifikation erklärt `iterator()` wie folgt. `T` ist dabei der Typ, über den später iteriert werden soll:

<code>Iterator<T> iterator()</code>	Returns an iterator over elements of type T.
---	--

`Iterator<T>` ist dabei selbst ein Interface. Dieses muss von einer eigenen Klasse implementiert werden. Wir implementieren es als geschachtelte Klasse innerhalb der `Vector4D`-Klasse. `Iterator<T>` erfordert die Implementierung der folgenden Methoden:

<code>boolean hasNext()</code>	Returns true if the iteration has more elements.
<code>T next()</code>	Returns the next element in the iteration.

Vervollständigen Sie auf den nächsten Seiten die Klassendefinitionen, deklarieren und implementieren Sie die `iterator()`-Methode und implementieren Sie die dazugehörige `Iterator`-Klasse namens `Vector4DIterator`.

Java Iterable Vector4D – 10 Punkte

```
1 // Klassendefinition, es fehlt die Implementierungsanweisung
2 public class Vector4D {
3     private Double x, y, z, w;
4
5     public Vector4D(Double x, Double y, Double z, Double w) {
6         this.x = x; this.y = y; this.z = z; this.w = w;
7     }
8
9     public Double getX() { return x; }
10    public void setX(Double x) { this.x = x; }
11    public Double getY() { return y; }
12    public void setY(Double y) { this.y = y; }
13    public Double getZ() { return z; }
14    public void setZ(Double z) { this.z = z; }
15    public Double getW() { return w; }
16    public void setW(Double w) { this.w = w; }
```

Java Iterable Vector4D – 10 Punkte

```
1 // Klassendefinition, es fehlt die Implementierungsanweisung
2 public class Vector4D implements Iterable<Double> {
3     private Double x, y, z, w;
4
5     public Vector4D(Double x, Double y, Double z, Double w) {
6         this.x = x; this.y = y; this.z = z; this.w = w;
7     }
8
9     public Double getX() { return x; }
10    public void setX(Double x) { this.x = x; }
11    public Double getY() { return y; }
12    public void setY(Double y) { this.y = y; }
13    public Double getZ() { return z; }
14    public void setZ(Double z) { this.z = z; }
15    public Double getW() { return w; }
16    public void setW(Double w) { this.w = w; }
```

Java Iterable Vector4D – 10 Punkte

```
1 // Klassendefinition, es fehlt die Implementierungsanweisung
2 public class Vector4D implements Iterable<Double> {
3     private Double x, y, z, w;
4
5     public Vector4D(Double x, Double y, Double z, Double w) {
6         this.x = x; this.y = y; this.z = z; this.w = w;
7     }
8
9     // [...]
10
11     // Implementieren Sie hier die iterator()-Methode
12     // Die geschachtelte Klasse heißt Vector4DIterator
13
14
15
16
17
18
19
20
21
22
23
```

Java Iterable Vector4D – 10 Punkte

```
1 // Klassendefinition, es fehlt die Implementierungsanweisung
2 public class Vector4D implements Iterable<Double> {
3     private Double x, y, z, w;
4
5     public Vector4D(Double x, Double y, Double z, Double w) {
6         this.x = x; this.y = y; this.z = z; this.w = w;
7     }
8
9     // [...]
10
11     // Implementieren Sie hier die iterator()-Methode
12     // Die geschachtelte Klasse heißt Vector4DIterator
13     @Override
14     public Iterator<Double> iterator() {
15         return new Vector4DIterator();
16     }
17
18
19
20
21
22
23 }
```

Java Iterable Vector4D – 10 Punkte

```
24 // die geschachtelte Klasse, es fehlt wieder die
25 // Implementierungsanweisung
26 private class Vector4DIterator {
27     /* TODO */
28
29
30
31
32
33
34
35
36
37
38
39
40 }
41 }
```

Java Iterable Vector4D – 10 Punkte

```
24 // Implementieren Sie hier die Methoden des Iterators;
25     private class Vector4DIterator implements Iterator<Double> {
26         private int next = 0;
27
28         @Override
29         public boolean hasNext() {
30             /* TODO */
31         }
32
33         @Override
34         public Double next() {
35             /* TODO */
36         }
37     }
38 }
39 }
```

Java Iterable Vector4D – 10 Punkte

```
26 private class Vector4DIterator implements Iterator<Double> {
27     int next = 0;
28     @Override
29     public boolean hasNext() {
30         return next < 4;
31     }
32     @Override
33     public Double next() {
34         switch (next++) {
35             case 0: return x;
36             case 1: return y;
37             case 2: return z;
38             case 3: return w;
39             default: return null;
40         }
41     }
42 }
43 }
```

siehe  **IntelliJ IDEA**

LÖSUNG PROGRAMMIERAUFGABE 4 CRYPTO

Fragestunde

Viel Erfolg bei der Klausur!